

홈랩 구축 트레이드오프 모음

월 왜 골랐는지 — 2026년 8월 정리

홈랩을 만드는 동안 "정석은 이거지만 지금 상황엔 이게 낫다"는 판단을 여러 번 했다. 나중에 다시 보거나 누가 이유를 물어봤을 때 바로 답할 수 있게 그때그때의 판단 근거를 모아뒀다.

방향성

홈랩 전체 컨셉

AI 인프라 + 플랫폼 엔지니어링을 하나로 묶음

GitOps, CI/CD, 서비스 메시, RAG, kagent까지 하고 싶은 게 많았는데, 이것들을 각각 따로 없으면 16GB로는 물리적으로 동시에 못 돌리고, "이것저것 넣어본 홈랩"이라는 인상만 남아서 하나의 이야기로 설명하기 어려워진다. 그래서 "AI 워크로드(RAG+llama.cpp)를 GitOps로 배포하는 셀프서비스 플랫폼을 운영한다"는 한 문장으로 목표를 좁히고, 플랫폼 엔지니어링을 방법론, AI 서빙을 그 플랫폼이 다루는 대표 워크로드로 역할을 나눴다.

RAG/LLAMA.CPP를 어떻게 취급할지

제품이 아니라 인프라 워크로드로

GTX 1060 3GB로는 RAG 답변 품질을 자랑할 수 있는 수준이 못 된다. 그래서 "똑똑한 챗봇을 만들었다"가 아니라 "제한된 GPU 리소스 안에서 이 워크로드를 어떻게 스케줄링하고 운영했는가"에 초점을 두기로 했다. 별도 LLM 웹사이트를 만드는 방향도 검토했지만, 이미 만들어둔 서버리스 게임 포트폴리오와 성격이 겹쳐서 이 홈랩만의 차별점(인프라 운영 경험)이 흐려질 것 같아 제외했다.

노드 / 클러스터 구성

VM 만드는 방식

cloud-init 템플릿 vs ISO 수동 설치

GUI로 마스터를 만들다가 설치 마법사 중간에 비밀번호를 깜빡해서 다시 지운 게 계기였다. ISO는 매번 마법사를 손으로 진행해야 하지만 cloud-init은 템플릿 하나 만들어두면 `qm clone` 한 줄로 끝난다. 3대를 반복해서 만들어야 하는 상황이라 cloud-init 쪽이 확실히 이득이었다.

쿠버네티스 배포 도구

kubeadm vs k3s

k3s가 더 가볍고 16GB 환경엔 유리하지만, 실무에서 더 많이 쓰이는 kubeadm으로 직접 구성하는 경험 자체를 이력에 남기고 싶어서 kubeadm을 선택했다. 대신 컨트롤플레인과 워커 리소스를 평소보다 빠듯하게 나눠 잡아야 했다.

CNI

Calico vs Flannel

Flannel은 가볍지만 NetworkPolicy를 지원하지 않는다. 나중에 서비스들끼리 트래픽을 제어하는 시나리오를 다룰 계획이 있어서, 처음부터 Calico로 구성했다. pod-network-cidr을 Calico 기본값(`192.168.0.0/16`) 그대로 쓰면 집 LAN 대역과 겹치는 걸 뒤늦게 발견해서 `10.244.0.0/16` 으로 바꿔 지정해야 했다.

쿠버네티스 버전

1.31 → 1.35로 정정

습관적으로 1.31을 쓰려다가 이미 EOL된 버전이라는 걸 확인했다. 최근 3개 마이너 버전만 지원되는데, 그중 가장 먼저 나와서 서드파티 애드온 호환성이 제일 검증된 1.35로 정했다.

서비스 메시 / 관측성

서비스 메시

Istio 대신 Linkerd, 혹은 Ambient 모드

PLG 스택(약 2GB)과 Istio(사이드카 포함 2GB 이상, 서비스가 늘수록 더 커짐)를 워커 2대(8GB) 안에서 동시에 상시로 돌리는 건 계산상 무리였다. Istio는 채용공고 노출 빈도가 높아 이름값은 있지만, 사이드카가 Envoy 대비 훨씬 가벼운 Linkerd로 대체하거나, Istio를 꼭 쓰려면 사이드카 없는 Ambient 모드를 쓰는 쪽으로 정리했다.

분산 스토리지

Longhorn 레플리카 3 → 2로 축소

기본값인 3중 복제는 저장 공간과 메모리를 그만큼 잡아먹는다. 지금 규모에서 진짜 3중 이중화까지는 필요 없다고 보고, 2개로 낮춰서 "복제되는 분산 스토리지"라는 개념은 유지하면서 부담은 줄이는 쪽으로 방향을 잡았다.

리소스

RAM 증설

8GB 대신 4GB, 속도보다 용량

DDR4 가격이 오른 시기라 8GB 추가는 부담스러워서 4GB로 낮췄다. 기존보다 느린 DDR4-2133도 검토했는데, 여러 모듈을 섞으면 전체가 가장 느린 모듈 기준으로 다운클럭된다는 걸 감안해도, VM 여러 개를 돌리는 용도에서는 대역폭보다 용량(OOM을 안 만나는 것)이 더 중요하다고 판단했다. 대역폭 손해는 대략 20% 수준으로 추정되지만 지금 워크로드에서는 체감이 크지 않을 걸로 본다.

CI/CD 서버 배치

Jenkins는 클러스터 파드 대신 별도 VM

리소스만 보면 k8s 파드로 띄우는 쪽이 훨씬 유리하다. 그런데도 별도 VM으로 만들기로 했는데, 실무에서 CI/CD 서버를 별도로 분리해서 운영하는 경우가 많다는 점을 직접 경험해보고 싶어서였다. 대신 최소 스펙(2vCPU/2GB)으로 잡아서 부담을 줄였다.

PROXMOX 확장

멀티 노드 클러스터링은 보류

Datacenter 밑에 노드를 여러 개 두려면 물리적으로 별개인 서버가 필요하다. 지금 PC 한 대뿐이라, Nested 가상화로 가짜 2번째 노드를 만드는 것도 검토했지만 실질적인 HA 효과 없이 RAM만 더 쓰는 선택이라 보류했다.

원격 접근 / 네트워킹

원격 접속 방식

Cloudflare Tunnel — Tailscale과는 용도 구분

Tailscale은 이미 많이 써봐서 이번엔 Cloudflare Tunnel로 새로 시도했다. 대체 관계가 아니라 용도가 다르다고 정리했다 — Tailscale은 기기 전체를 사설망 하나로 묶는 방식이라 나중에 AWS와의 하이브리드 연결(Phase 5)에 남겨두고, Cloudflare Tunnel은 특정 웹 서비스 몇 개만 도메인으로 열어주는 지금 목적에 더 맞았다.

CLOUDFLARED 설치 위치

전용 VM/파드 대신 Proxmox 호스트에 직접

원칙적으로는 하이퍼바이저 호스트에 애플리케이션을 얹지 않는 게 맞다 — 보안 환경이 커지고, 호스트에 문제가 생기면 그 위의 VM 전부가 영향을 받는다. k8s 파드로 띄우는 방법이 리소스 측면에서는 제일 나았지만, 이미 호스트에 apt 저장소까지 등록하고 진행 중이라 그대로 마무리했다. cloudflared 자체가 아웃바운드 전용의 가벼운 데몬이라 상대적으로 리스크가 작다고 보고 감수했다.